

Part 1: Exploratory Data Analysis (EDA) - 10 points

Task 1.1: Load and examine the dataset

Import Libraries

```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    roc_curve,
    classification_report,
    roc_curve,
    roc_auc_score
)
from sklearn.ensemble import RandomForestClassifier
from itertools import combinations
import statsmodels.api as sm
import statsmodels.formula.api as smf
from statsmodels.stats.outliers_influence import
variance_inflation_factor
from scipy import stats
```

Loading the Dataset, Checking the Data Shape, and Displaying the Data types and the First Few Rows

```
# Load the dataset downloaded from Kaggle
heart = pd.read_csv("heart.csv")

# Display data shape, types, and summary statistics
print("\nDataset Information:")
```

```

print(heart.info())
print(heart.describe())

# Missing values
print("\nMissing Values:")
print(heart.isnull().sum())

# First 5 rows
display(heart.head())

```

Dataset Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 303 entries, 0 to 302

Data columns (total 14 columns):

#	Column	Non-Null Count	Dtype
0	age	303 non-null	int64
1	sex	303 non-null	int64
2	cp	303 non-null	int64
3	trestbps	303 non-null	int64
4	chol	303 non-null	int64
5	fbs	303 non-null	int64
6	restecg	303 non-null	int64
7	thalach	303 non-null	int64
8	exang	303 non-null	int64
9	oldpeak	303 non-null	float64
10	slope	303 non-null	int64
11	ca	303 non-null	int64
12	thal	303 non-null	int64
13	target	303 non-null	int64

dtypes: float64(1), int64(13)

memory usage: 33.3 KB

None

	age	sex	cp	trestbps	chol
fbs \					
count	303.000000	303.000000	303.000000	303.000000	303.000000
mean	54.438944	0.679868	2.158416	131.689769	246.693069
std	9.038662	0.467299	0.960126	17.599748	51.776918
min	29.000000	0.000000	0.000000	94.000000	126.000000
25%	48.000000	0.000000	2.000000	120.000000	211.000000
50%	56.000000	1.000000	2.000000	130.000000	241.000000
75%	61.000000	1.000000	3.000000	140.000000	275.000000

max	77.000000	1.000000	3.000000	200.000000	564.000000
1.000000					

	restecg	thalach	exang	oldpeak	slope
ca \					
count	303.000000	303.000000	303.000000	303.000000	303.000000
303.000000					
mean	0.990099	149.607261	0.326733	1.039604	0.600660
0.663366					
std	0.994971	22.875003	0.469794	1.161075	0.616226
0.934375					
min	0.000000	71.000000	0.000000	0.000000	0.000000
0.000000					
25%	0.000000	133.500000	0.000000	0.000000	0.000000
0.000000					
50%	1.000000	153.000000	0.000000	0.800000	1.000000
0.000000					
75%	2.000000	166.000000	1.000000	1.600000	1.000000
1.000000					
max	2.000000	202.000000	1.000000	6.200000	2.000000
3.000000					

	thal	target
count	303.000000	303.000000
mean	1.831683	0.458746
std	0.956705	0.499120
min	1.000000	0.000000
25%	1.000000	0.000000
50%	1.000000	0.000000
75%	3.000000	1.000000
max	3.000000	1.000000

Missing Values:

age	0
sex	0
cp	0
trestbps	0
chol	0
fbs	0
restecg	0
thalach	0
exang	0
oldpeak	0
slope	0
ca	0
thal	0
target	0

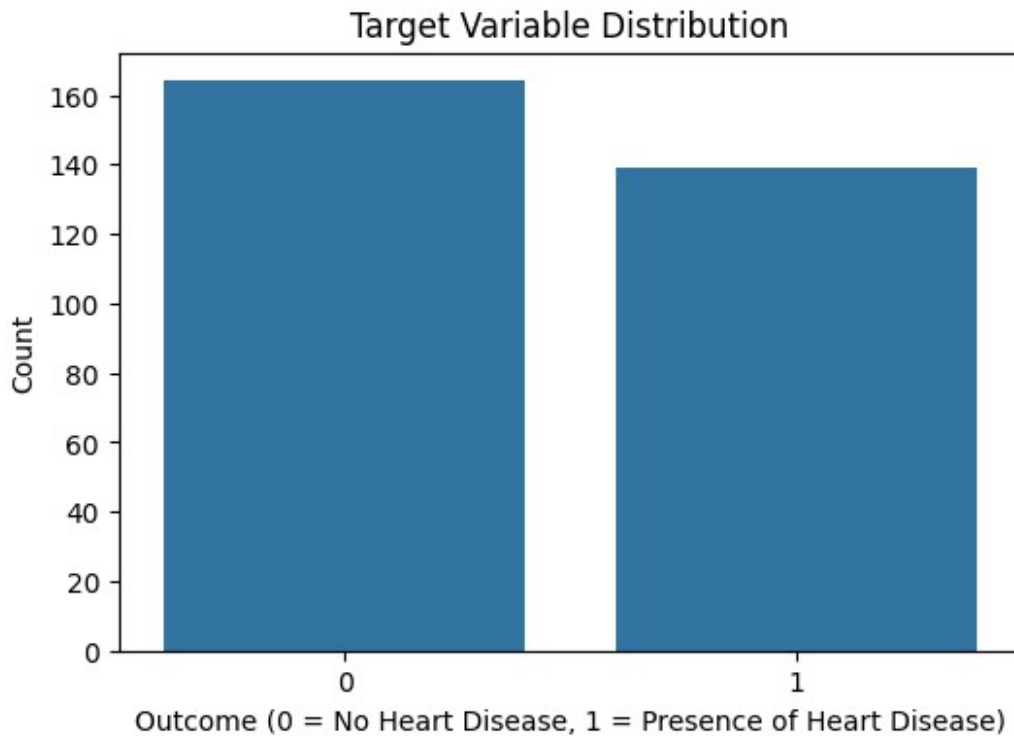
dtype: int64

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak
slope \										
0	63	1	0	145	233	1	2	150	0	2.3
2										
1	67	1	3	160	286	0	2	108	1	1.5
1										
2	67	1	3	120	229	0	2	129	1	2.6
1										
3	37	1	2	130	250	0	0	187	0	3.5
2										
4	41	0	1	130	204	0	2	172	0	1.4
0										
	ca	thal	target							
0	0	2	0							
1	3	1	1							
2	2	3	1							
3	0	1	0							
4	0	1	0							

Bar Graph for Checking Class Imbalance in the 'Target' Variable

Check Class Imbalance in 'Target' using Matplotlib Bar Plot

```
plt.figure(figsize=(6,4))
sns.countplot(x="target", data=heart)
plt.title("Target Variable Distribution")
plt.xlabel("Outcome (0 = No Heart Disease, 1 = Presence of Heart Disease)")
plt.ylabel("Count")
plt.show()
```



Observation

- *Based on the Target Variable Distribution, the amount of observations for each class in the target variable is sufficient and suggests that there is no class imbalance present in the dataset.*

Task 1.2: Visualize distributions

Reshaping Data into Long Categorical and Long Continuous Format

```
# Continuous variables
numeric_cols = ['age', 'trestbps', 'chol', 'thalach', 'oldpeak']
# Categorical variables
categ_cols = ['sex', 'cp', 'fbs', 'restecg', 'exang', 'slope', 'ca',
              'thal']

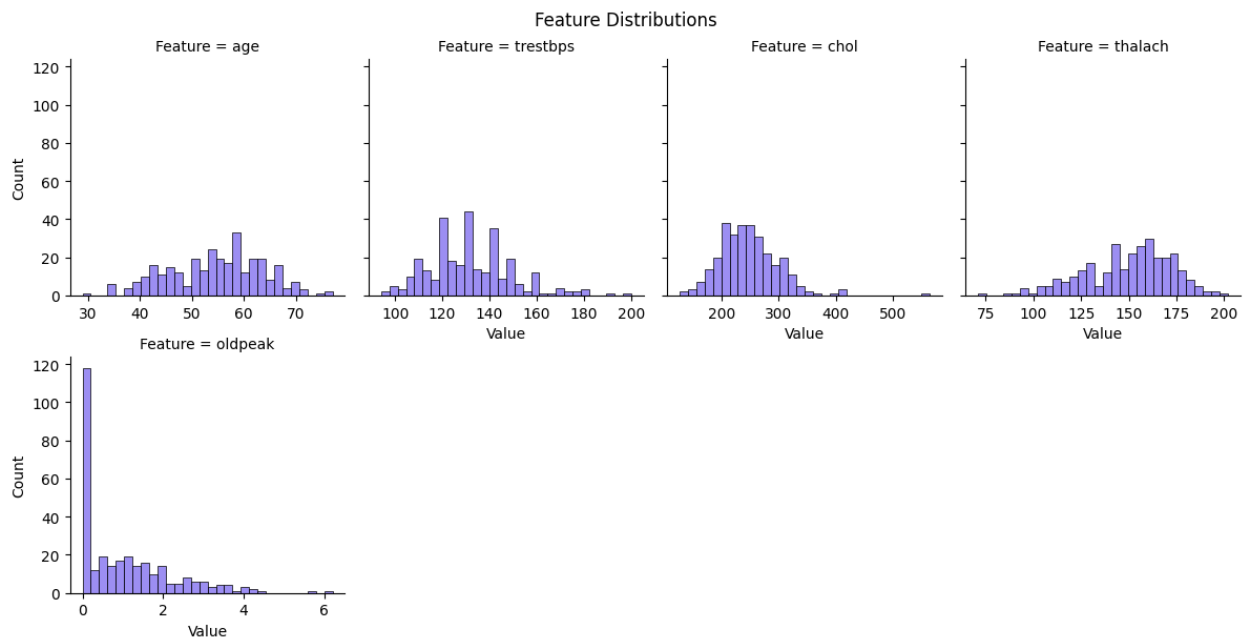
# Reshaping the Data for Continuous Variables
heart_long_numeric = heart.melt(
    id_vars="target",
    value_vars=numeric_cols,
    var_name="Feature",
    value_name="Value"
)

# Reshaping the Data for Categorical Variables
```

```
heart_long_cat = heart.melt(
    id_vars="target",
    value_vars=categ_cols,
    var_name="Feature",
    value_name="Value"
)
```

Histograms for Continuous Variables

```
g = sns.FacetGrid(heart_long_numeric, col="Feature", col_wrap=4,
    sharex=False)
g.map(sns.histplot, "Value", bins=30, color="mediumslateblue")
g.fig.suptitle("Feature Distributions", y=1.02)
plt.show()
```



Bar Plots for Categorical Variables

```
import seaborn as sns
import matplotlib.pyplot as plt

g = sns.FacetGrid(
    heart_long_cat,
    col="Feature",
    col_wrap=4,
    sharex=False,
    sharey=False
)

g.map_dataframe(
```

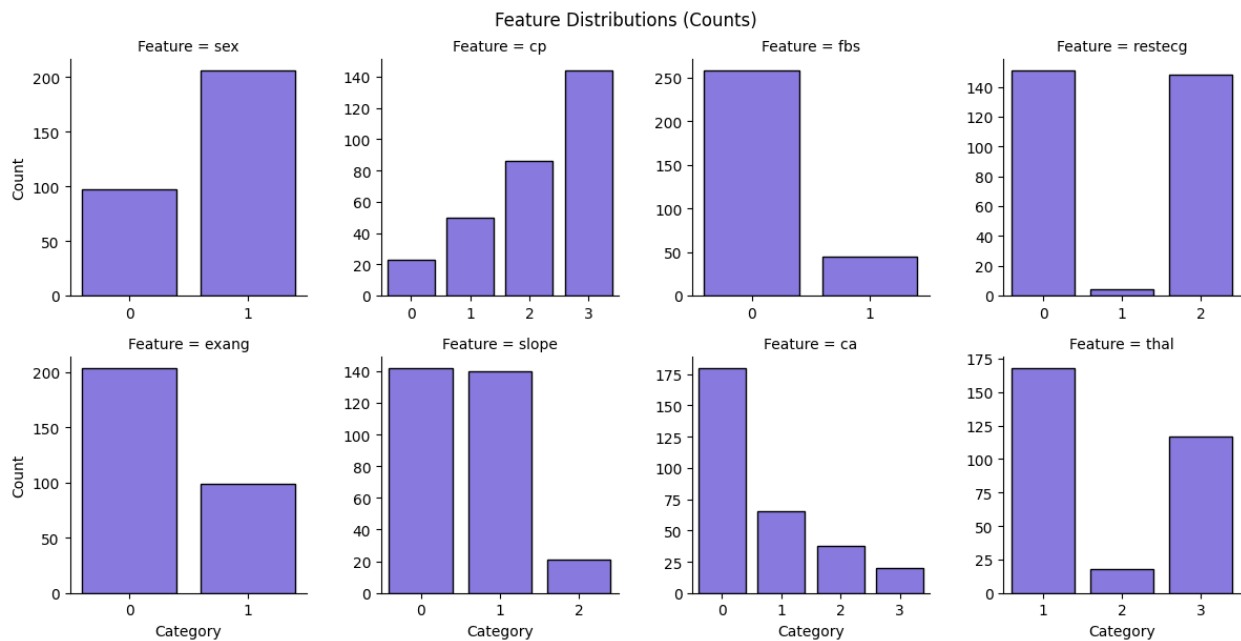
```

sns.countplot,
x="Value",
color="mediumslateblue",
edgecolor="black",
linewidth=1
)

g.fig.suptitle("Feature Distributions (Counts)", y=1.02)
g.set_axis_labels("Category", "Count")

plt.show()

```



Count Plot for Target Variable

```

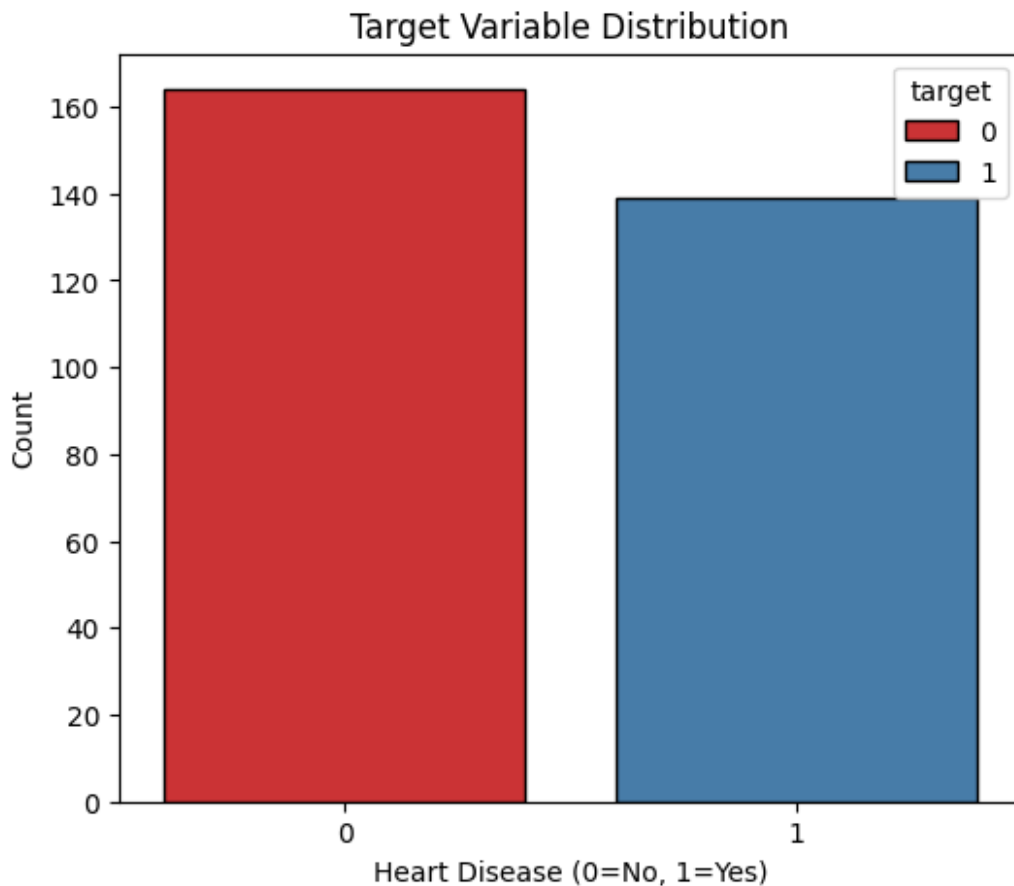
plt.figure(figsize=(6, 5))

# Create a single count plot for the target
sns.countplot(
    data=heart,
    x="target",
    hue="target",
    palette="Set1",
    edgecolor="black",
    linewidth=1
)

plt.title("Target Variable Distribution")
plt.xlabel("Heart Disease (0=No, 1=Yes)")

```

```
plt.ylabel("Count")  
plt.show()
```



Identifying potential outliers using boxplots:

- Based on the histograms for the continuous predictors ('age', 'trestbps', 'chol', 'thalach', and 'oldpeak'), there are no potential outliers detected since the distribution of data leans toward a normal distribution. As for the categorical predictors ('sex', 'cp', 'fbs', 'restecg', 'exang', 'slope', 'ca', and 'thal'), the bar plots indicate that data is well-distributed and balanced for some of the variables, however, for predictors 'restecg' and 'thal', some classes have very few observations which may affect model building.

Task 1.3: Explore relationships with target variable

Side-by-Side Boxplots of Continuous Predictors by Target class

```
g = sns.FacetGrid(  
    heart_long_numeric,
```



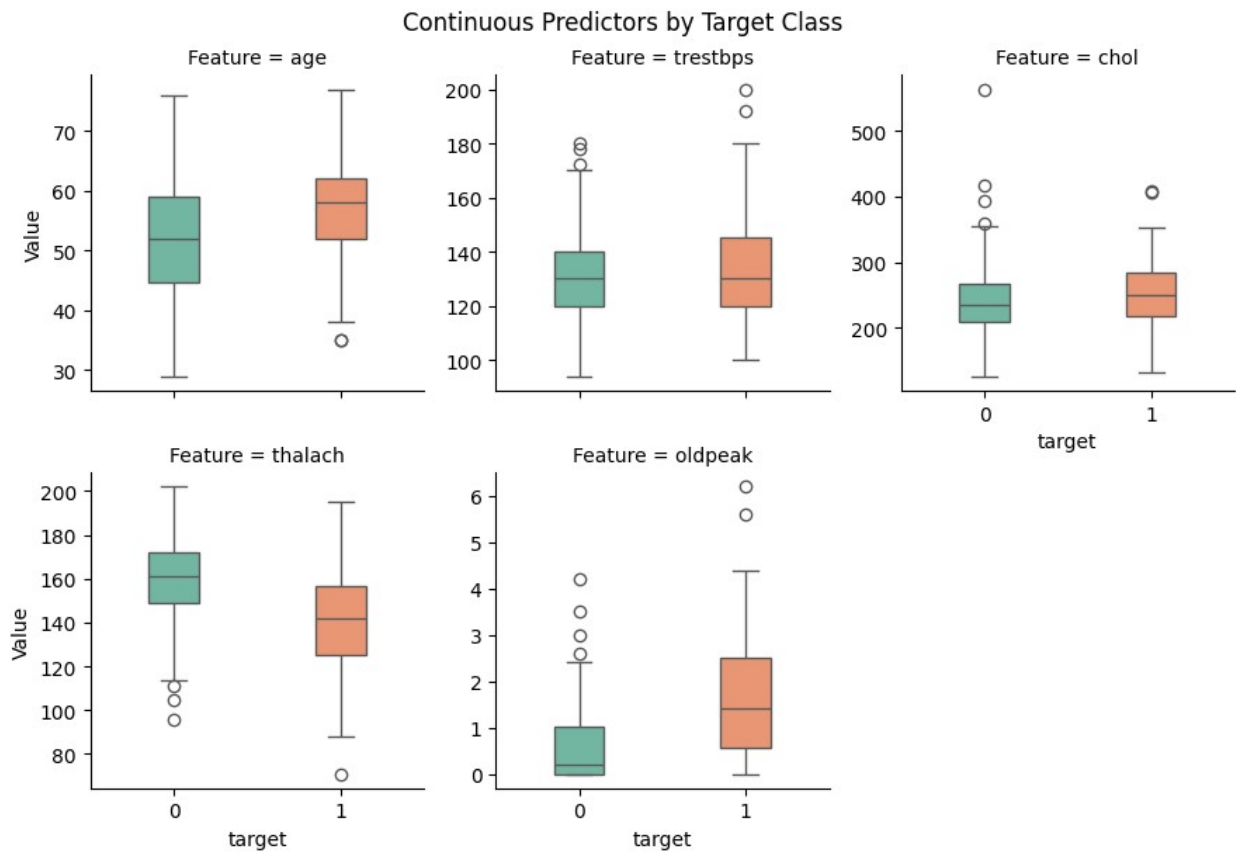
```

col="Feature",
col_wrap=3,
sharey=False
)

g.map_dataframe(
    sns.boxplot,
    x="target",
    y="Value",
    hue="target",
    palette="Set2",
    width=0.3,
    legend=False
)

g.fig.suptitle("Continuous Predictors by Target Class", y=1.02)
plt.show()

```



Stacked Bar Charts for Categorical Predictors by Target Class

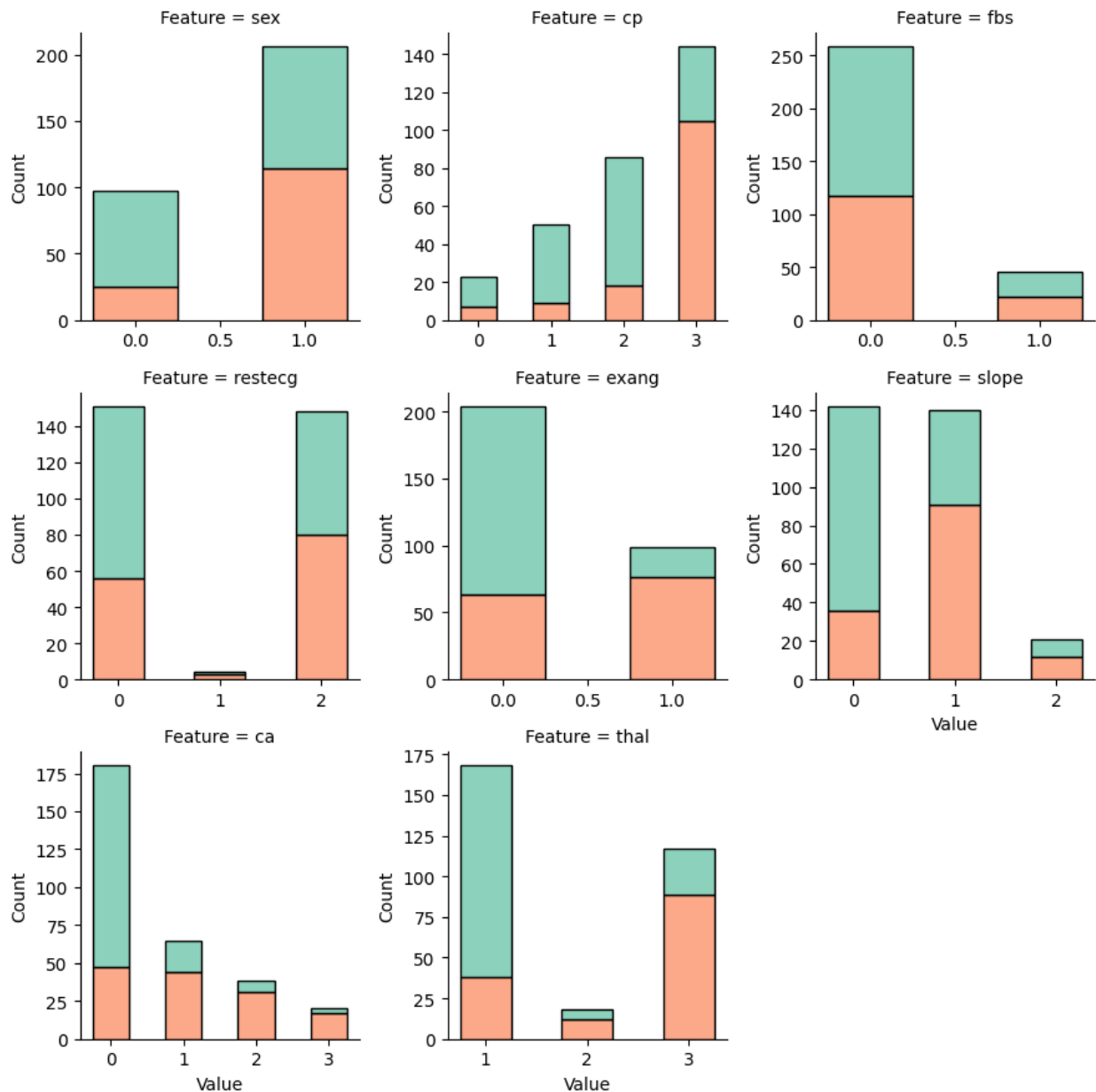
```

g = sns.FacetGrid(
    heart_long_catg,
    col="Feature",

```

```
        col_wrap=3,  
        sharey=False,  
        sharex=False  
    )  
    g.map_dataframe(  
        sns.histplot,  
        x="Value",  
        hue="target",  
        multiple="stack",  
        palette="Set2",  
        discrete=True,  
        shrink=0.5,  
        legend=True  
    )  
    g.fig.suptitle("Categorical Predictors by Target Class", y=1.02)  
    plt.show()
```

Categorical Predictors by Target Class



Observation:

- For continuous predictors, 'oldpeak' and 'thalach' show clear differences between classes since the data distribution for each class are clustered at different values. As for the categorical predictors, 'cp', 'exang', 'slope', 'ca', and 'thal' also show clear differences between classes since most of the some of their respective categories are heavily dominated by a class.

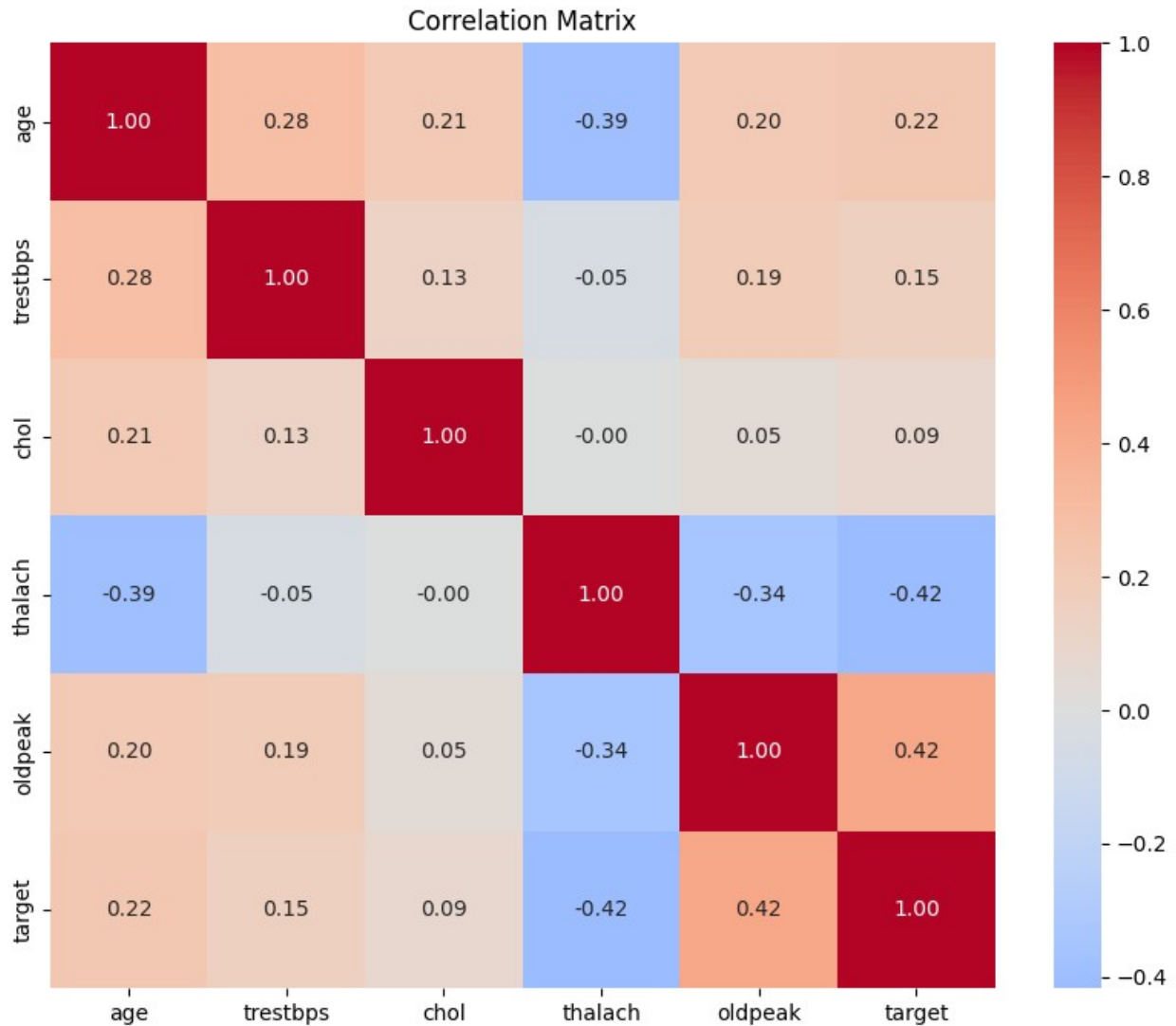
Part 2: Correlation Analysis - 10 points

Task 2.1: Compute and visualize correlation matrix

```
corr_data = ['age', 'trestbps', 'chol', 'thalach', 'oldpeak',  
             'target']  
  
corr_matrix = heart[corr_data].corr()  
print("\nCorrelation Matrix for Numeric Variables:")  
print(corr_matrix)  
  
plt.figure(figsize=(10,8))  
sns.heatmap(  
    corr_matrix,  
    cmap="coolwarm",  
    center=0,  
    square=True,  
    annot=True,  
    fmt=".2f"  
)  
plt.title("Correlation Matrix")  
plt.show()
```

Correlation Matrix for Numeric Variables:

	age	trestbps	chol	thalach	oldpeak	target
age	1.000000	0.284946	0.208950	-0.393806	0.203805	0.223120
trestbps	0.284946	1.000000	0.130120	-0.045351	0.189171	0.150825
chol	0.208950	0.130120	1.000000	-0.003432	0.046564	0.085164
thalach	-0.393806	-0.045351	-0.003432	1.000000	-0.343085	-0.417167
oldpeak	0.203805	0.189171	0.046564	-0.343085	1.000000	0.424510
target	0.223120	0.150825	0.085164	-0.417167	0.424510	1.000000



Task 2.2: Interpret correlations

```
# Finding variables with high correlation
# Marker for High Correlation
threshold = 0.7

high_corr_pairs = []

# Creates the list of highly correlated variable pairs (absolute value
# >= threshold)
for i in range(len(corr_matrix.columns)):
    for j in range(i + 1, len(corr_matrix.columns)):
        col_1 = corr_matrix.columns[i]
```

```

col_2 = corr_matrix.columns[j]
val = corr_matrix.iloc[i, j]

if abs(val) >= threshold:
    high_corr_pairs.append((col_1, col_2, val))

print(f"Variables with correlation >= {threshold} (or <= -
{threshold}):")
print("-" * 50)
for col1, col2, val in high_corr_pairs:
    print(f"{col1} vs {col2}: {val:.2f}")

Variables with correlation >= 0.7 (or <= -0.7):
-----

```

Observation:

- *Based on the correlational matrix and the heatmap created, there are no numeric predictors that are strongly correlated with the target variable. Upon observation, there are no correlation coefficient between predictors that exceeded the threshold (> 0.7 or < -0.7).*

Part 3: Full Logistic Regression Model - 10 points

Task 3.1: Build the full model

```

# Feature Scaling

scaler = StandardScaler()
X_scaled = scaler.fit_transform(
    heart.drop(columns="target")
)

heart_scaled = pd.DataFrame(
    X_scaled,
    columns=heart.columns[:-1]
)
heart_scaled["target"] = heart["target"].values

# Logistic Regression (Full Model - ALL predictors)
X = heart_scaled.drop(columns="target")
y = heart_scaled["target"]

# Add constant
X_fm = sm.add_constant(X)

```

```
full_model = sm.Logit(y, X_fm).fit()
full_model.summary()
```

```
Optimization terminated successfully.
      Current function value: 0.343483
      Iterations 7
```

```
<class 'statsmodels.iolib.summary.Summary'>
"""
```

Logit Regression Results

```
=====
Dep. Variable:          target    No. Observations:
303
Model:                  Logit     Df Residuals:
289
Method:                 MLE       Df Model:
13
Date:                  Fri, 20 Feb 2026    Pseudo R-squ.:
0.5020
Time:                  20:46:56    Log-Likelihood:
-104.08
converged:              True      LL-Null:
-208.99
Covariance Type:        nonrobust    LLR p-value:
1.301e-37
=====
```

```
=====
              coef      std err          z      P>|z|      [0.025
0.975]
-----
const      -0.0835      0.185      -0.452      0.651      -0.445
0.279
age        -0.1289      0.216      -0.597      0.551      -0.553
0.295
sex         0.6411      0.227       2.822      0.005       0.196
1.086
cp          0.5890      0.183       3.222      0.001       0.231
0.947
trestbps    0.4208      0.188       2.237      0.025       0.052
0.790
chol        0.2511      0.195       1.288      0.198      -0.131
0.633
fbs        -0.2676      0.188      -1.420      0.156      -0.637
0.102
restecg     0.2352      0.182       1.294      0.196      -0.121
0.591
=====
```

thalach	-0.4674	0.233	-2.006	0.045	-0.924
-0.011					
exang	0.4789	0.192	2.489	0.013	0.102
0.856					
oldpeak	0.2874	0.244	1.176	0.240	-0.192
0.767					
slope	0.3694	0.221	1.673	0.094	-0.063
0.802					
ca	1.1538	0.245	4.700	0.000	0.673
1.635					
thal	0.6465	0.190	3.409	0.001	0.275
1.018					

```
=====
=====
" " "
```

Report:

- The Log-likelihood of the full model is -104.08, while the Null Log-likelihood is -208.99. Since there is a significant increase from the LL-null to LL, this indicates that the addition of the 13 predictors contributes to explaining the target variable, and using the full model fits the data significantly better.

- Null Deviance** = $2(\text{ll}(\text{Saturated Model}) - \text{ll}(\text{Null Model}))$

- Residual Deviance** = $2(\text{ll}(\text{Saturated Model}) - \text{ll}(\text{Proposed Model}))$

====> **Null Deviance: 417.98** / **Residual Deviance: 208.16** ====> The Residual Deviance is significantly lower than the Null Deviance, indicating that including the 13 predictors in the model significantly reduced the discrepancy in the model and improved the model's ability to classify the target.

Task 3.2: Initial model interpretation

```
# Determining which predictors are statistically significant at  $\alpha = 0.05$ 
```

```
full_model.pvalues
```

const	0.651255
age	0.550734
sex	0.004767
cp	0.001275
trestbps	0.025307
chol	0.197593
fbs	0.155620
restecg	0.195581
thalach	0.044872


```
exang      0.012805
oldpeak    0.239785
slope      0.094420
ca         0.000003
thal       0.000652
dtype: float64
```

Observation

- Upon observing the p-values of the predictors, only 'sex', 'cp', 'trestbps', 'thalach', 'exang', 'ca', 'thal' are statistically significant, satisfying $p \leq 0.05$
- The Likelihood ratio test value is $1.301e-37$, which implies that the full model better fits the data significantly as compared to the null model with only the constant. Thus, the overall model is statistically significant, wherein at least one of the 13 predictors contributes meaningfully in classifying the target.

Part 4: Linearity Assumption - Log Odds - 10 points

Task 4.1: Test linearity of continuous predictors to log odds For each continuous predictor, test the linearity assumption using one of these methods:

Box-Tidwell Test

```
# Continuous Predictors to include in Box-Tidwell transformation
numeric_cols = ['age', 'trestbps', 'chol', 'thalach', 'oldpeak']

# Duplicate copy of Predictor DataFrame for Box-Tidwell transformation
X_bt = X.copy()

# Add Box-Tidwell terms (X * log(X))
epsilon = 1e-5

for col in numeric_cols:
    X_bt[f"{col}_log"] = X_bt[col] * np.log(np.abs(X_bt[col]) +
epsilon)

# Gather continuous predictors and the log terms for the model
predictors = [item for col in numeric_cols for item in (col,
```

```
f"{col}_log")]
```

```
# Fit the Logistic Regression Model
```

```
bt_model = sm.Logit(
```

```
    y,
```

```
    sm.add_constant(X_bt[predictors])
```

```
).fit()
```

```
print(bt_model.summary())
```

```
Optimization terminated successfully.
```

```
Current function value: 0.511708
```

```
Iterations 7
```

Logit Regression Results

```
=====
```

```
=====
```

Dep. Variable:	target	No. Observations:	
303			
Model:	Logit	Df Residuals:	
292			
Method:	MLE	Df Model:	
10			
Date:	Fri, 20 Feb 2026	Pseudo R-squ.:	
0.2581			
Time:	20:46:56	Log-Likelihood:	
-155.05			
converged:	True	LL-Null:	
-208.99			
Covariance Type:	nonrobust	LLR p-value:	
1.422e-18			

```
=====
```

```
=====
```

	coef	std err	z	P> z	[0.025
					0.975]

const	-0.2243	0.160	-1.402	0.161	-0.538
0.089					
age	0.2990	0.189	1.578	0.115	-0.072
0.670					
age_log	-0.8566	0.338	-2.534	0.011	-1.519
-0.194					
trestbps	0.0010	0.188	0.005	0.996	-0.367
0.369					
trestbps_log	0.3873	0.290	1.337	0.181	-0.180
0.955					
chol	0.4328	0.181	2.392	0.017	0.078
0.788					
chol_log	-0.5316	0.265	-2.010	0.044	-1.050

-0.013					
thalach	-0.7738	0.193	-4.007	0.000	-1.152
-0.395					
thalach_log	-0.0841	0.345	-0.244	0.808	-0.761
0.593					
oldpeak	0.8169	0.186	4.381	0.000	0.451
1.182					
oldpeak_log	0.1669	0.359	0.465	0.642	-0.537
0.871					
=====					
=====					

Task 4.2: Document findings

- There are three continuous variables that satisfy the linearity assumption ($p \geq 5$), these are trestbps, thalach, and oldpeak.
- The continuous variables that violates the linearity assumption, on the other hand, are age and chol, since they are statistically significant at $p \leq 0.05$

Part 5: Multicollinearity Assessment - 10 points

Task 5.1: Calculate Variance Inflation Factor (VIF)

```
# Check VIF for multicollinearity
vif_data = pd.DataFrame()
vif_data["Feature"] = X.columns
vif_data["VIF"] = [
    variance_inflation_factor(X.values, i)
    for i in range(X.shape[1])
]
```

vif_data

	Feature	VIF
0	age	1.492629
1	sex	1.283312
2	cp	1.317185
3	trestbps	1.197041
4	chol	1.145477
5	fbs	1.073243

6	restecg	1.088768
7	thalach	1.624956
8	exang	1.385712
9	oldpeak	1.762107
10	slope	1.674063
11	ca	1.352483
12	thal	1.466615

Task 5.2: Interpret and address multicollinearity

- The VIF table shows that there are no multicollinearity present between the predictors. The VIF values of the predictors hovered around 1.5, which suggests a very low to no multicollinearity, since the values that suggest a moderate to high multicollinearity is 5-10.

Part 6: Goodness of Fit - Hosmer-Lemeshow Test - 10 points

Task 6.1: Perform Hosmer-Lemeshow test on full model

Hosmer-Lemeshow Test

```
def hosmer_lemeshow(y_true, y_prob, g=10):
    data = pd.DataFrame({"y": y_true, "p": y_prob})
    data["group"] = pd.qcut(data["p"], g)

    obs = data.groupby("group", observed=False)["y"].sum()
    exp = data.groupby("group", observed=False)["p"].sum()
    n = data.groupby("group", observed=False)["y"].count()

    hl_stat = np.sum((obs - exp)**2 / (exp * (1 - exp/n)))
    p_value = 1 - stats.chi2.cdf(hl_stat, g-2)

    return hl_stat, p_value

# Perform Hosmer-Lemeshow test on the full model
hl_stat, p_value = hosmer_lemeshow(
    y,
    full_model.predict(X_fm)
)
```

```
print("HL Statistic:", hl_stat)
print("p-value:", p_value)
```

```
HL Statistic: 4.708098029447281
p-value: 0.7882706000865538
```

Calibration Plot

```
# Get predicted probabilities
y_prob = full_model.predict(X_fm)

# Create dataframe
calib_df = pd.DataFrame({
    "y_true": y,
    "y_prob": y_prob
})

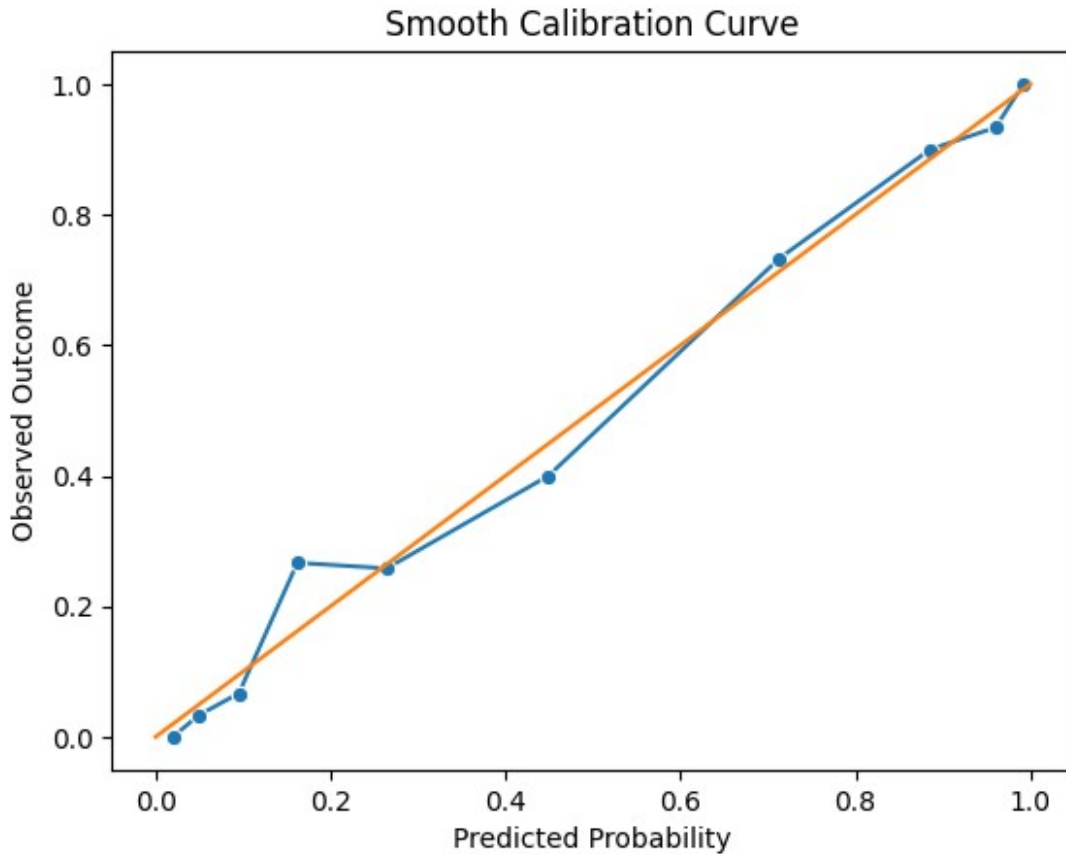
# Create probability bins
calib_df["bin"] = pd.qcut(calib_df["y_prob"], 10, duplicates="drop")

# Compute observed and expected probabilities per bin
calibration = calib_df.groupby("bin", observed=False).agg(
    observed_rate=("y_true", "mean"),
    expected_rate=("y_prob", "mean")
).reset_index()

# Plot
plt.figure()

sns.lineplot(
    data=calibration,
    x="expected_rate",
    y="observed_rate",
    marker="o"
)

plt.plot([0, 1], [0, 1])
plt.xlabel("Predicted Probability")
plt.ylabel("Observed Outcome")
plt.title("Smooth Calibration Curve")
plt.show()
```



Task 6.2: Interpret the results

- With an HL statistic p-value of 0.788, exceeding the threshold 0.05, this indicates that there is no significant evidence to assume a difference between the predicted probabilities and the observed event frequencies.
- Upon seeing the calibration plot, there are no major systematic patterns observed. The lineplot does not significantly deviate from the reference line, indicating good calibration with no systematic bias. Several fluctuations from the line were observed, however, this does not overall imply a systematic bias.
- In terms of the full model's predictive ability, the calibration plot shows that the model's predicted probabilities are well-calibrated and highly reliable.

Part 7: Alternative Model Building - Method 1 - 10 points

Task 7.1: Build your first alternative model
Choose one method:

Best Subset (BIC) Selection

```
def best_subset_selection_bic(X, y):  
    features = X.columns.tolist()  
    n_features = len(features)  
  
    # Total combinations must be 2^N - 1  
    total_combos = (2 ** n_features) - 1  
  
    # Initialize output variables  
    best_bic = float('inf')  
    best_features = None  
  
    for k in range(1, n_features + 1):  
        for combo in combinations(features, k):  
            current_combo = list(combo)  
            try:  
                X_subset = sm.add_constant(X[current_combo])  
  
                model = sm.Logit(y, X_subset).fit(dis=False)  
  
                if model.bic < best_bic:  
                    best_bic = model.bic  
                    best_features = current_combo  
            except Exception:  
                continue  
  
    # Display the best model and features  
    print("\nBest Model (BIC): ")  
    print(f"Lowest BIC: {best_bic:.2f}")  
    print(f"Number of Features: {len(best_features)}")  
    print(f"Best Features: {best_features}")  
  
    return best_features, model  
  
# Perform best subset selection using BIC  
subset_features, best_model= best_subset_selection_bic(X, y)
```

```
# Rebuild the dataset one last time using ONLY the winning features
X_sub = sm.add_constant(X[subset_features])

# Train the absolute best model one final time so we can give it to
the user
subset_model = sm.Logit(y, X_sub).fit(dis=False)

print("\nBest Model Summary:")
print(subset_model.summary())
```

```
Best Model (BIC):
Lowest BIC: 264.32
Number of Features: 5
Best Features: ['cp', 'exang', 'oldpeak', 'ca', 'thal']
```

Best Model Summary:

Logit Regression Results

```
=====
=====
Dep. Variable:                target    No. Observations:
303
Model:                        Logit     Df Residuals:
297
Method:                       MLE      Df Model:
5
Date:                        Fri, 20 Feb 2026    Pseudo R-squ.:
0.4496
Time:                        20:48:40    Log-Likelihood:
-115.02
converged:                    True      LL-Null:
-208.99
Covariance Type:             nonrobust    LLR p-value:
1.076e-38
=====
=====
```

	coef	std err	z	P> z	[0.025
0.975]					

const	-0.0589	0.172	-0.343	0.731	-0.395
0.277					
cp	0.5649	0.173	3.261	0.001	0.225
0.904					
exang	0.5536	0.176	3.138	0.002	0.208
0.899					
oldpeak	0.6936	0.202	3.435	0.001	0.298
1.089					
ca	1.0351	0.204	5.070	0.000	0.635

1.435					
thal	0.8820	0.169	5.208	0.000	0.550
1.214					

```
=====
=====
```

Task 7.2: Document your approach

- The method chosen for this alternative model is the best subset selection, since it tests and iterates through all possible combinations of predictors, ensuring that the predictors selected have a meaningful contribution in explaining the data.
- The selection criterion used is BIC (Bayesian Information Criterion), as it penalizes the addition of insignificant predictors to the model, ensuring that the model is resistant to overfitting.
- The variables that were selected by the algorithm are 'cp', 'exang', 'oldpeak', 'ca', and 'thal', since these variables yielded the lowest BIC values.

Part 8: Alternative Model Building - Method 2 - 10 points

Task 8.1: Build your second alternative model *Choose a DIFFERENT method from Part 7:*

```
import pandas as pd
import statsmodels.api as sm
from sklearn.ensemble import RandomForestClassifier

# Random Forest for Feature Importance (much more stable and resistant
# than Decision Trees)
''' the parameters used below are selected since they are the most
widely and
        commonly used for building a strong baseline model '''

rf_model = RandomForestClassifier(
    n_estimators=100,
    max_depth=7,
    min_samples_leaf=5,
    max_features='sqrt',
    class_weight='balanced',
    n_jobs=-1,
    random_state=143
)
```

```

rf_model.fit(X, y)

# Ranking Predictors using Feature Importances from the Random Forest model
importance_df = pd.DataFrame({
    'Feature': X.columns,
    'Importance': rf_model.feature_importances_
})
importance_df = importance_df.sort_values(by='Importance',
ascending=False)

print("Ranked Predictors by Feature Importances from Random Forest:")
print(importance_df)

# Calculate threshold for filtering most important features using Mean Importance
threshold = 1.0 / len(X.columns)

# Finalize list of selected features based on importance threshold
selected_features = importance_df[importance_df['Importance'] >
threshold]['Feature'].tolist()

print(f"\nSelected {len(selected_features)} features for Logistic Regression: {selected_features}\n")

# Building the Logistic Regression model using the RF-selected features
X_rf = sm.add_constant(X[selected_features])

# Train the final Logistic Regression model
rf_logit_model = sm.Logit(y, X_rf).fit()

# View the results
print(rf_logit_model.summary())

```

Ranked Predictors by Feature Importances from Random Forest:

	Feature	Importance
2	cp	0.192365
12	thal	0.169731
11	ca	0.133301
7	thalach	0.100590
9	oldpeak	0.099419
8	exang	0.069769
0	age	0.056880
10	slope	0.047924
4	chol	0.042944
3	trestbps	0.038375
1	sex	0.033258
6	restecg	0.013775

5 fbs 0.001669

Selected 5 features for Logistic Regression: ['cp', 'thal', 'ca', 'thalach', 'oldpeak']

Optimization terminated successfully.

Current function value: 0.385701

Iterations 7

Logit Regression Results

```
=====
=====
Dep. Variable:                target    No. Observations:
303
Model:                        Logit     Df Residuals:
297
Method:                       MLE      Df Model:
5
Date:                         Fri, 20 Feb 2026    Pseudo R-squ.:
0.4408
Time:                         20:48:41    Log-Likelihood:
-116.87
converged:                    True      LL-Null:
-208.99
Covariance Type:              nonrobust    LLR p-value:
6.624e-38
=====
=====
```

	coef	std err	z	P> z	[0.025
0.975]					

const	-0.0734	0.171	-0.430	0.667	-0.408
0.261					
cp	0.6187	0.171	3.620	0.000	0.284
0.954					
thal	0.9249	0.167	5.534	0.000	0.597
1.252					
ca	0.9872	0.205	4.809	0.000	0.585
1.389					
thalach	-0.4535	0.185	-2.448	0.014	-0.817
-0.090					
oldpeak	0.6769	0.199	3.397	0.001	0.286
1.067					

Task 8.2: Document your approach

- The method chosen for the second alternative model is the random forest-selected logistic regression, since it is much more stable and robust than decision tree-based feature importance ranking. This method is an ensemble-learning model, wherein the output of many ensemble learners are averaged.
- Once the random forest was trained, the feature importance of each predictor was evaluated on the threshold $1.0 / \text{len}(X.\text{columns})$ in order to filter out predictors that are assumed to be insignificant.
- The method yielded only five significant predictors, these are: 'cp', 'thal', 'ca', 'thalach', 'oldpeak'

Part 9: Model Evaluation and Comparison - 10 points

Task 9.1: Split data for evaluation

```
# Create train-test split 80:20
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=143, stratify=y)

# Check class distribution in train set
print(y_train.value_counts().reset_index())

# Check class distribution in test set
print(y_test.value_counts().reset_index())
```

	target	count
0	0	131
1	1	111

	target	count
0	0	33
1	1	28

Task 9.2: Calculate performance metrics for all three models For each model (Full Model, Method 1, Method 2), calculate on TEST data:

Full Model

```

X_train_fm = sm.add_constant(X_train)
X_test_fm = sm.add_constant(X_test)

full_model1 = sm.Logit(y_train, X_train_fm).fit()

pred_prob = full_model1.predict(X_test_fm)
pred_class = (pred_prob > 0.5).astype(int)

print("Full Model Confusion Matrix:")
print(confusion_matrix(y_test, pred_class))

print("\nFull Model Classification Report:")
print(classification_report(y_test, pred_class))

Optimization terminated successfully.
    Current function value: 0.336802
    Iterations 7
Full Model Confusion Matrix:
[[30  3]
 [10 18]]

Full Model Classification Report:

```

	precision	recall	f1-score	support
0	0.75	0.91	0.82	33
1	0.86	0.64	0.73	28
accuracy			0.79	61
macro avg	0.80	0.78	0.78	61
weighted avg	0.80	0.79	0.78	61

Best Subset (BIC) Model

```

X_train_subset = sm.add_constant(X_train[subset_features])
X_test_subset = sm.add_constant(X_test[subset_features])

subset_model1 = sm.Logit(y_train, X_train_subset).fit()
pred_prob2 = subset_model1.predict(X_test_subset)
pred_class2 = (pred_prob2 > 0.5).astype(int)

print("Best Subset (BIC) Confusion Matrix:")
print(confusion_matrix(y_test, pred_class2))

print("\nBest Subset (BIC) Classification Report:")
print(classification_report(y_test, pred_class2))

Optimization terminated successfully.
    Current function value: 0.366811
    Iterations 7

```

Best Subset (BIC) Confusion Matrix:

```
[[31  2]
 [11 17]]
```

Best Subset (BIC) Classification Report:

	precision	recall	f1-score	support
0	0.74	0.94	0.83	33
1	0.89	0.61	0.72	28
accuracy			0.79	61
macro avg	0.82	0.77	0.78	61
weighted avg	0.81	0.79	0.78	61

Random Forest Selected Logistic Regression

```
X_train_rf = sm.add_constant(X_train[selected_features])
X_test_rf = sm.add_constant(X_test[selected_features])
```

```
rf_logit_model1 = sm.Logit(y_train, X_train_rf).fit()
pred_prob3 = rf_logit_model1.predict(X_test_rf)
pred_class3 = (pred_prob3 > 0.5).astype(int)
```

```
print("Random Forest Selected Confusion Matrix:")
print(confusion_matrix(y_test, pred_class3))
```

```
print("\nRandom Forest Selected Classification Report:")
print(classification_report(y_test, pred_class3))
```

```
Optimization terminated successfully.
      Current function value: 0.373638
      Iterations 7
```

Random Forest Selected Confusion Matrix:

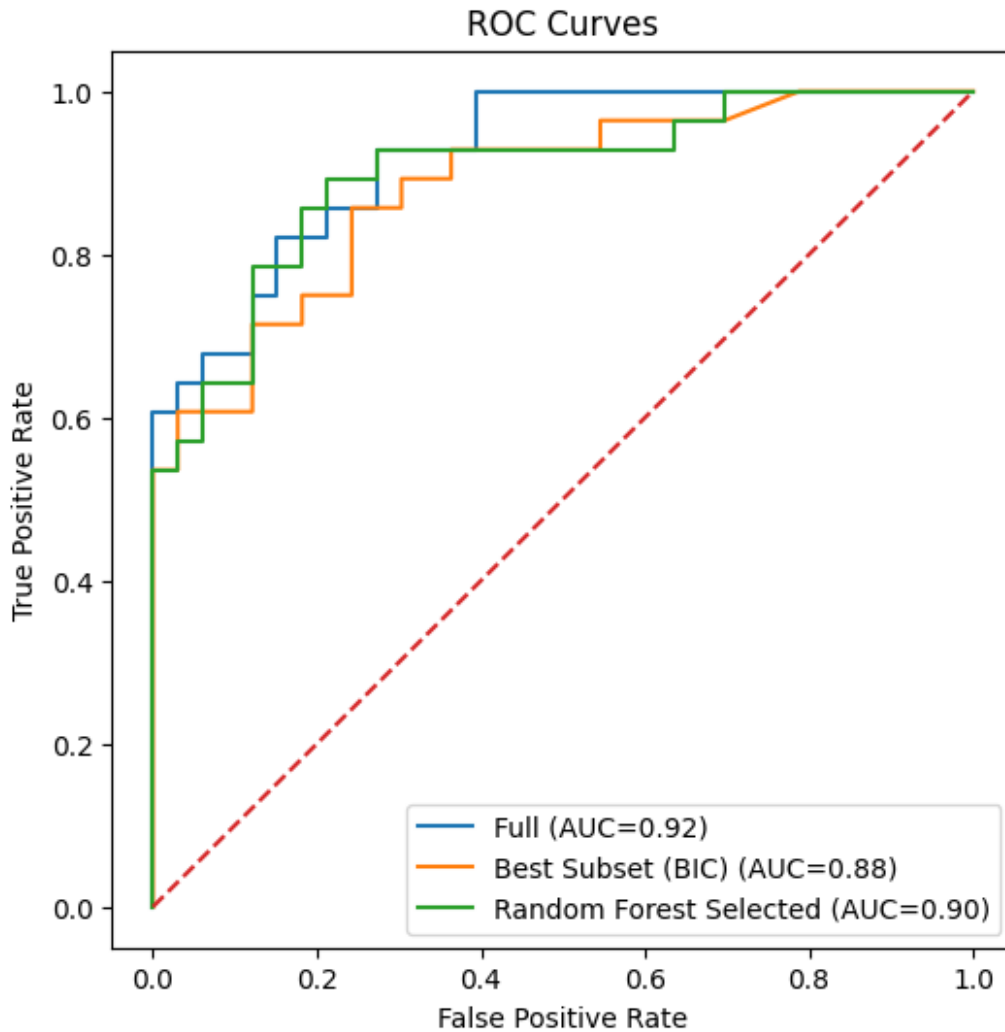
```
[[29  4]
 [10 18]]
```

Random Forest Selected Classification Report:

	precision	recall	f1-score	support
0	0.74	0.88	0.81	33
1	0.82	0.64	0.72	28
accuracy			0.77	61
macro avg	0.78	0.76	0.76	61
weighted avg	0.78	0.77	0.77	61

Task 9.3: Create visualizations

```
def plot_roc(y_true, y_prob, label):  
    fpr, tpr, _ = roc_curve(y_true, y_prob)  
    auc = roc_auc_score(y_true, y_prob)  
    plt.plot(fpr, tpr, label=f"{label} (AUC={auc:.2f})")  
  
plt.figure(figsize=(6,6))  
  
plot_roc(y_test, full_model.predict(X_test_fm), "Full")  
plot_roc(y_test, subset_model.predict(X_test_subset), "Best Subset  
(BIC)")  
plot_roc(y_test, rf_logit_model.predict(X_test_rf), "Random Forest  
Selected")  
  
plt.plot([0,1], [0,1], linestyle="--")  
plt.xlabel("False Positive Rate")  
plt.ylabel("True Positive Rate")  
plt.legend()  
plt.title("ROC Curves")  
plt.show()
```



Task 9.4: Build comparison table

```
import pandas as pd

# Create Comparison Table for Performance Metrics based on the
# classification reports and AUC scores
performance_data = {
    "Model": [
        "Random Forest",
        "Best Subset (BIC)",
        "Full Model"
    ],
    "Accuracy": [
        0.77,
        0.79,
        0.79
    ]
}
```



```

    ],
    "Precision": [
        0.78,
        0.81,
        0.80
    ],
    "Recall": [
        0.77,
        0.79,
        0.79
    ],
    "F1 Score": [
        0.77,
        0.78,
        0.78
    ],
    "AUC": [
        0.90,
        0.88,
        0.92
    ]
}

# Create dataframe
performance_df = pd.DataFrame(performance_data)

print(performance_df)

```

	Model	Accuracy	Precision	Recall	F1 Score	AUC
0	Random Forest	0.77	0.78	0.77	0.77	0.90
1	Best Subset (BIC)	0.79	0.81	0.79	0.78	0.88
2	Full Model	0.79	0.80	0.79	0.78	0.92

Task 9.5: Select the best model

- Out of all the models, the Full Model performed best, obtaining the highest AUC while maximizing accuracy and balanced precision and recall. However, the performance of the Random Forest tracks closely to the Full model, with significantly few predictors, effectively addressing the issue of parsimony
- There are trade-offs between the metrics, since the BIC model achieved the highest precision, but taking into account of all the metrics, the Full Model is the most balanced performance out of all three.
- Therefore, upon considering the performances of each model while taking into account of model parsimony, the final model chosen is the **Random Forest Selected Logistic Regression Model**.

Part 10: Best Model Interpretation - 10 points

Task 10.1: Interpret coefficients of the best model For each predictor in your chosen best model:

```
# Obtaining the Coefficients of Random Forest Selected Model
coefficients = rf_logit_model1.params
print(coefficients)
```

```
# Getting the Odds Ratios
odds_ratios = np.exp(coefficients)
```

```
odds_table = pd.DataFrame({
    "Coefficient ( $\beta$ )": coefficients,
    "Odds Ratio ( $e^{\beta}$ )": odds_ratios
}).reset_index().rename(columns={"index": "Variable"})
```

```
odds_table
```

```
const      -0.198416
cp           0.540767
thal        0.923880
ca           0.943127
thalach     -0.574316
oldpeak      0.639432
dtype: float64
```

	Variable	Coefficient (β)	Odds Ratio (e^{β})
0	const	-0.198416	0.820029
1	cp	0.540767	1.717323
2	thal	0.923880	2.519046
3	ca	0.943127	2.567999
4	thalach	-0.574316	0.563090
5	oldpeak	0.639432	1.895405

Observation:

Continuous Variables:

- thalach - A one-unit increase in thalach is associated with a 0.56 times change in odds in the odds of heart disease.
- oldpeak - A one-unit increase in ST depression induced by exercise relative to rest is associated with an 1.86 times change in odds of heart disease.

Binary Variables:

- cp - Patients with Chest pain type 1 has 1.72 times the odds of heart disease compared to patients without
- thal - Patients with Fixed defect Thalassemia has 2.52 times the odds of heart disease compared to Normal Patients.
- ca - Patients with 1 major vessel colored has 2.57 times the odds of heart disease compared to Patients without colored major vessels.

Task 10.2 - 10.3: Wald Test for statistical significance and Interpretation Table

```
# Use rf_logit_model1 for Wald statistics and p-values

# Filter Predictors that are statistically significant at  $\alpha = 0.05$ 
alpha = 0.05
significance = np.where(rf_logit_model1.pvalues < alpha,
                        "Significant", "Not Significant")

# Summary table
summary_table = pd.DataFrame({
    "Predictor": rf_logit_model1.params.index,
    "Coefficient ( $\beta$ )": rf_logit_model1.params.values,
    "Std Error": rf_logit_model1.bse.values,
    "Wald Statistic": rf_logit_model1.tvalues.values,
    "p-value": rf_logit_model1.pvalues.values,
    "Odds Ratio ( $e^{\beta}$ )": np.exp(rf_logit_model1.params.values),
    "Interpretation": significance
})

print(summary_table)
```

	Predictor	Coefficient (β)	Std Error	Wald Statistic	p-value \
0	const	-0.198416	0.195176	-1.016602	0.309343
1	cp	0.540767	0.192808	2.804685	0.005037
2	thal	0.923880	0.190278	4.855425	0.000001
3	ca	0.943127	0.229096	4.116737	0.000038
4	thalach	-0.574316	0.228872	-2.509333	0.012096
5	oldpeak	0.639432	0.220101	2.905181	0.003670

	Odds Ratio (e^{β})	Interpretation
0	0.820029	Not Significant
1	1.717323	Significant
2	2.519046	Significant
3	2.567999	Significant
4	0.563090	Significant
5	1.895405	Significant

Task 10.4: Summarize key findings

- The Top 3 predictors that displayed the strongest association with heart disease are **'thal', 'ca', and 'cp'**. These predictors yielded the lowest p-values (0.000001, 0.000038, 0.005037) among all of the predictors. In terms of the Wald's Statistics, these three also demonstrated the highest significance, with a bit of a trade-off between cp and old_peak, since old_peak yielded the third highest Wald's statistic.
- The direction of effects is consistent with medical knowledge since Thalassemia, Chest Pains, and the number of vessels colored by fluoroscopy are key indicators that are being used by doctors in real life. These factors are examined as part of the patient's medical history and history of present illnesses.

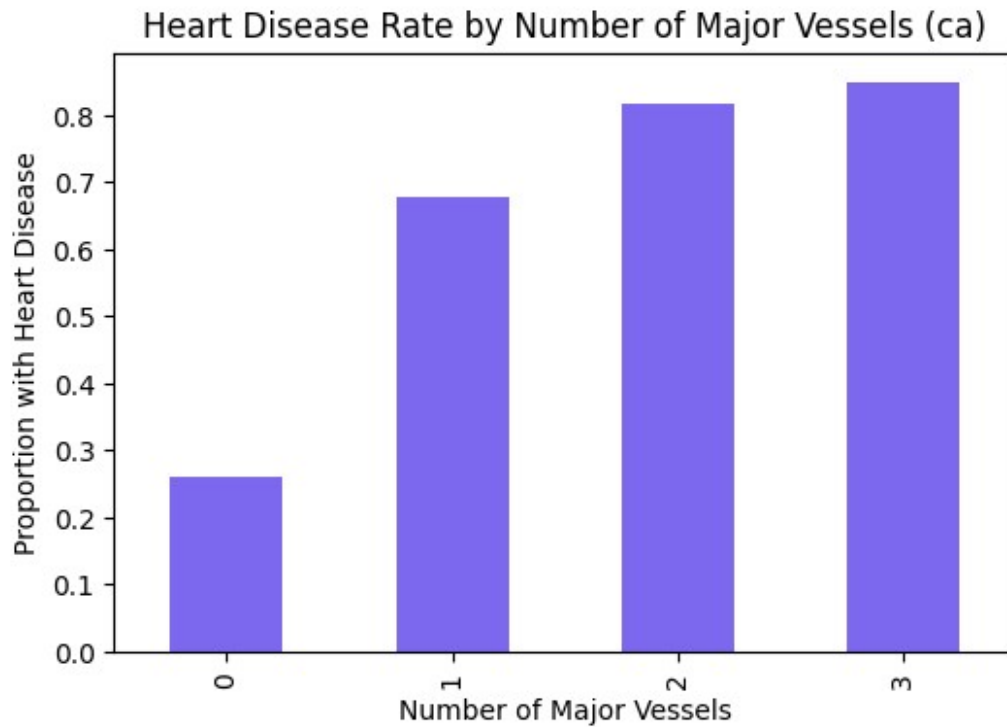
Part 11: Executive Summary for Non-Technical Audience

Key Risk Factors Identified

- After building a predictive model that determines patients with high-risk in heart diseases, three key risk factors are identified. It was discovered that patients with experiencing Chest Pains, have Thalassemia, and have major vessels colored by fluoroscopy, have a high risk of developing heart diseases or may currently have one. The impact of these factors were carefully examined to ensure its association with heart diseases. Patients with major blood vessels identified by fluoroscopy are 2.57 times more likely to have heart diseases, while patients with Thalassemia are 2.52 times more likely to be diagnosed with heart diseases. Patients with chest pains, on the other hand, are 1.72 times more likely to get heart diseases.

Visualization of Heart Disease Rate by Number of Major Vessels (ca)

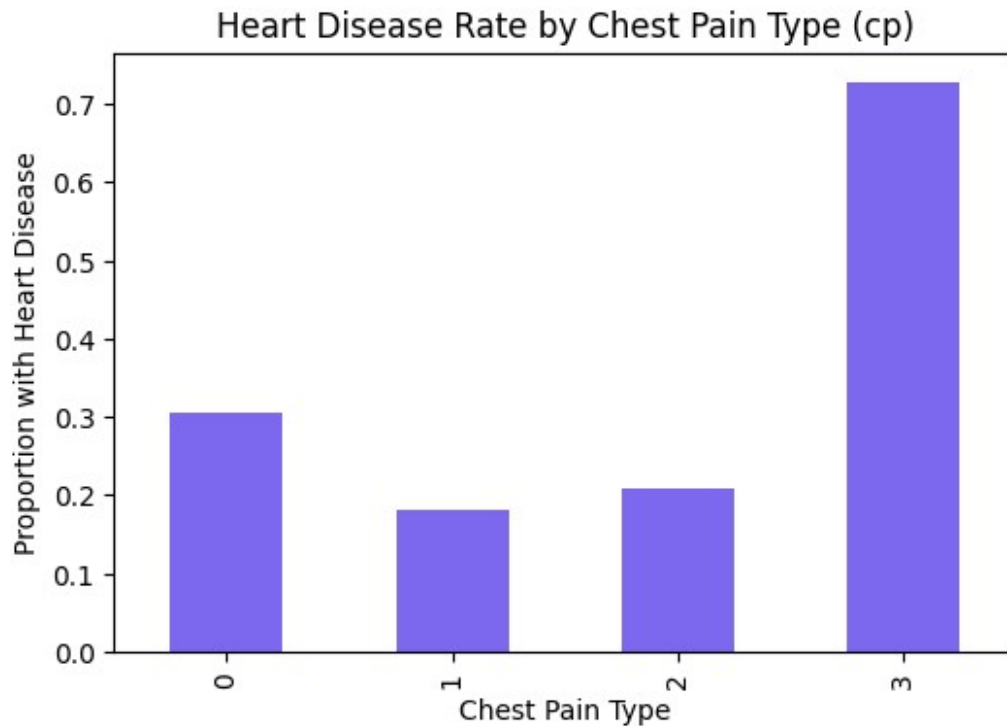
```
heart_ca = heart.groupby("ca")["target"].mean()  
  
plt.figure(figsize=(6,4))  
heart_ca.plot(kind="bar", color= "mediumslateblue")  
plt.title("Heart Disease Rate by Number of Major Vessels (ca)")  
plt.xlabel("Number of Major Vessels")  
plt.ylabel("Proportion with Heart Disease")  
plt.show()
```



Visualization of Heart Disease rate by Number of Major Vessels (cp)

```
heart_cp = heart.groupby("cp")["target"].mean()

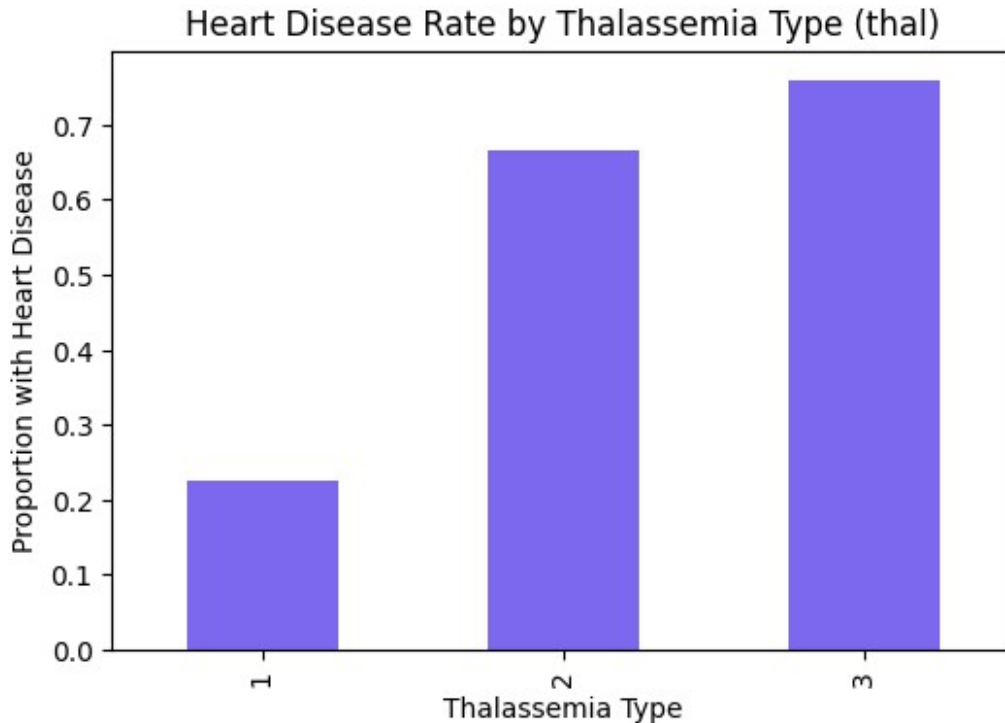
plt.figure(figsize=(6,4))
heart_cp.plot(kind="bar", color= "mediumslateblue")
plt.title("Heart Disease Rate by Chest Pain Type (cp)")
plt.xlabel("Chest Pain Type")
plt.ylabel("Proportion with Heart Disease")
plt.show()
```



Visualization of Heart Disease rate by Thalassemia

```
heart_thal = heart.groupby("thal")["target"].mean()

plt.figure(figsize=(6,4))
heart_thal.plot(kind="bar", color= "mediumslateblue")
plt.title("Heart Disease Rate by Thalassemia Type (thal)")
plt.xlabel("Thalassemia Type")
plt.ylabel("Proportion with Heart Disease")
plt.show()
```



Model Explanation (Non-Technical)

- The predictive model that was developed has one major objective, which is to classify patients that most likely have heart diseases based on a given set of demographic information and medical history. The impact of the factors to the risk-assessment based on patient information were heavily examined before including them into the predictive model in order to produce more accurate results. The factors that yielded the most importance in evaluating the risk-assessment of patients are Thalassemia, Chest pains, Number of major vessels colored by fluoroscopy, ST depression induced by exercise relative to rest, and the Maximum heart rate achieved. With these factors, the model achieved the highest accuracy among all of the candidate models that were trialed and tested.

Model Performance in Plain Language

- The model correctly identifies heart disease risk in 77% of patients overall. More importantly, it is excellent at distinguishing between patients with heart disease and those without. The model also yielded a relatively low false negative rate, implying that there is a low possibility that it can misclassify a patient as 'healthy' even if they actually have a heart disease. If a patient is misclassified as healthy instead of being diagnosed with heart disease/s, their current condition may continue to worsen without proper treatment and guidance. Since the model also displayed a significantly low false positive rate, this means that there is a low possibility that it can misdiagnose a patient with heart disease even if they do not actually have one. If this occurs, this incident may negatively impact the mental and emotional state of the patient misdiagnosed, while also going through costly treatments and consults. Since the model accurately ruled out more

healthy patients instead of correctly detecting patients with heart disease, this model may be used by health professionals for ruling out healthy patients.

Clinical Recommendation

- Overall, as a result of building the predictive model, 5 out of 13 factors were identified as key risk factors for detecting heart disease. These factors are Thalassemia, Chest pains, Number of major vessels colored by fluoroscopy, ST depression induced by exercise relative to rest, and the Maximum heart rate achieved. It is advised for medical practitioners to pay more attention to these factors/symptoms when evaluating a patient's medical history and present illnesses. Since the model demonstrated a good performance in identifying healthy patients, it can be used by professionals as a screening tool in order to improve efficiency in patient evaluation. However, the predictions made by the model are merely estimates, so it is important to support these estimates with human-based judgement. The rate of false negatives, however, should be focused on in future research, so in the meantime, medical practitioners should be extra cautious in relying on the negative estimates of the model.